

Installing the server software

We'll start by installing the database server, and then securing it. We'll first need to add the MariaDB repository, and the associated GPG key, to the package-management system. If you want to use vanilla MySQL, skip these steps, and install the *mysql-server* and *mysql-client* packages instead of the *mariadb* counterparts. Anyway, to go ahead with MariaDB, first type the following command:

```
sudoedit /etc/sources.list.d/mariadb.list
```

In the file that opens up, type in the following:

```
deb http://mirrors.xmission.com/mariadb/repo/5.1/ubuntu lucid
main
deb-src http://mirrors.xmission.com/mariadb/repo/5.1/ubuntu lucid
main
```

Since Ubuntu 10.04.2 uses MySQL 5.1 by default, introducing 5.2 into the system is going to break some ABIs. For this, we'll have to use the 5.1 branch of MariaDB, which is based on MySQL 5.1.

Now close *nano* by pressing *Ctrl+X*, and then *Y* to save. Now to add the GPG key, type the following:

```
sudo apt-key adv --recv-keys --keyserver keyserver.ubuntu.com
18B943DB
```

Right, so we've got the infrastructure set up. Now, use the code given below:

```
sudo apt-get update
sudo apt-get install mariadb-server mariadb-client
```

These packages 'provide' *mysql-server* and *mysql-client*, so that won't mess up your package manager. Great! We have a DB server on the system. However, this is a production server, so just having a server will not do; we'll have to secure this thing. Type in:

```
mysql_secure_installation
```

Run through this script. Once you do that, the server will be secure enough to run on a production system. Your database server should be all set; however, if you're running vanilla MySQL there's room for more optimisations. Open up */etc/mysql/my.cnf*, and add in the following single line:

```
skip-innodb
```

Once this is done, restart the MySQL server, and continue to the next stage.

The HTTP server and PHP interpreter

Type in:

```
sudo apt-get install php5-cgi php5-mysql apache2 libapache2-mod-
php5
```

This will install the Apache Web server and the PHP 5 interpreter. You'll now be able to install WordPress and configure a network of sites.

Installing WordPress

Assuming you've now unpacked the WordPress tarball into Apache's web root (*/var/www* by default), you should be able to start with the initial set-up of WordPress. You'll have to set up a normal WordPress site before you can enable Network, so let's start with the initial set-up now. First, we'll have to prepare MySQL, by creating a database and assigning credentials to it, so that WordPress can set itself up. Type in:

```
sudo mysql -u root -p
```

Type your MySQL root password. You'll now be dropped into the MySQL shell. To create the database and assign a user to it, type in the following:

```
CREATE DATABASE wp_network;
GRANT ALL PRIVILEGES ON wp_network.* TO wp_user@localhost
IDENTIFIED BY 'wp_password';
FLUSH PRIVILEGES;
quit;
```

The last statement will quit the MySQL shell. Remember to change the database name, user name and password to something else, for security's sake.

Now, open up *http://localhost* in a Web browser, and step through the WordPress installation wizard. When you're asked for database details, ensure you use the credentials that you gave when you created the database in the MySQL shell. Do not even think about using the *root* credentials—you'll risk getting your entire server cracked that way.

Now that you have the WordPress admin user details, test basic WordPress functionality. If it works, it's time to move on to the big thing: enabling Network.

WordPress Network

WordPress Network functionality depends on *htaccess* files, and *mod_rewrite*. Due to its complete reliance on *htaccess* it is very difficult to get a WordPress network running on servers other than Apache. Anyway, there are two ways in which WordPress 3.0 Networks can be defined: Subdirectory and Subdomains.